Q4 - Conway's Game of Life

Dean Yockey

My computer

My personal computer is a Lenovo ThinkCentre I bought for $75 on eBay. It's running the Linux Mint operating system. When running `lscpu`, the first few lines read:

```
Architecture:             x86_64
  CPU op-mode(s):         32-bit, 64-bit
  Address sizes:          39 bits physical, 48 bits virtual
  Byte Order:             Little Endian
CPU(s):                   4
  On-line CPU(s) list:    0-3
```

My computer has 4 CPUs.

conway.c

Command Line Arguments

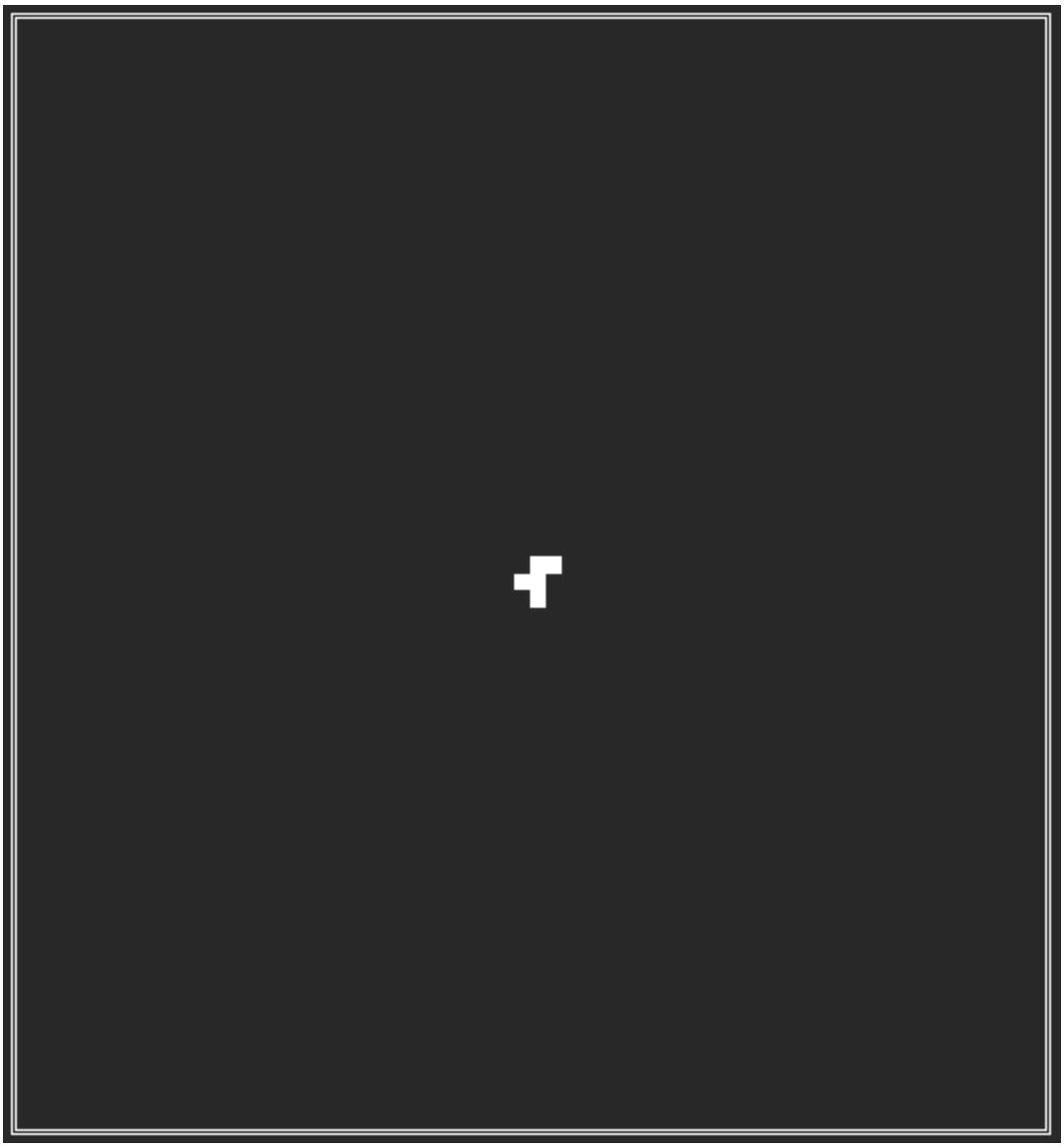
The program takes several arguments. The first two, for board size and number of generations, are mandatory. After this, the user can set a few command line options.

- `-D` disables all printing, aside from the final time reporting, and is used when benchmarking.
- `-r` redraws the board in the same position.
- `-p` causes the program to pause after every generation until the user presses enter. If the user enters `q`, the program will quit.

If the user enters `0` for the number of generations, the program will continue indefinitely, but the pause feature will automatically be turned on, like if `-p` was entered.

Starting State

The board starts out empty aside from an r-pentomino in the middle, which has a shape like:



This shape was chosen because it creates very a chaotic result from a very simple starting state.

Block Decomposition

The program then determines how to arrange processes' board subsections in rows and columns.

If the number of processes is a square number, `n^2`, then processes will be arranged in an even checkerboard pattern with `n` rows and `n` columns.

Otherwise, if the number of processes is a composite number, `n*m`, where `n > m`, the processes will be arranged in blocks, with `n` rows and `m` columns. `n` is greater than `m` because we prefer to have more rows than columns, making a process's subsection wider than it is tall, instead of the reverse. This is because the matrix is stored row-wise, and wider, shorter subsections means we switch rows less often, resulting in better cache usage.

Otherwise, if the number of processes is a prime number, `n`, the processes will be arranged in `n` rows. Thus it will devolve into block-row decomposition, instead of checkerboard decomposition.

Splitting and joining

The entire board is stored in process 0, and it is split into sub-boards which are sent to all other processes. After each process processes its sub-board it sends the new board (representing the state of the next generation) back to process 0. Process 0 then rejoins all these sub-boards into the full board, and then draws it (if drawing is enabled). Then, it splits the board again, starting the next generation.

The sub-boards that process 0 sends to other processes (called "in boards") are larger than the modified boards sent back (called "out boards"), unless running with just 1 process. This is because each process must read 1 more row and column in each direction than it writes, because each cell must check all of its neighbors to determine its next state.

Drawing

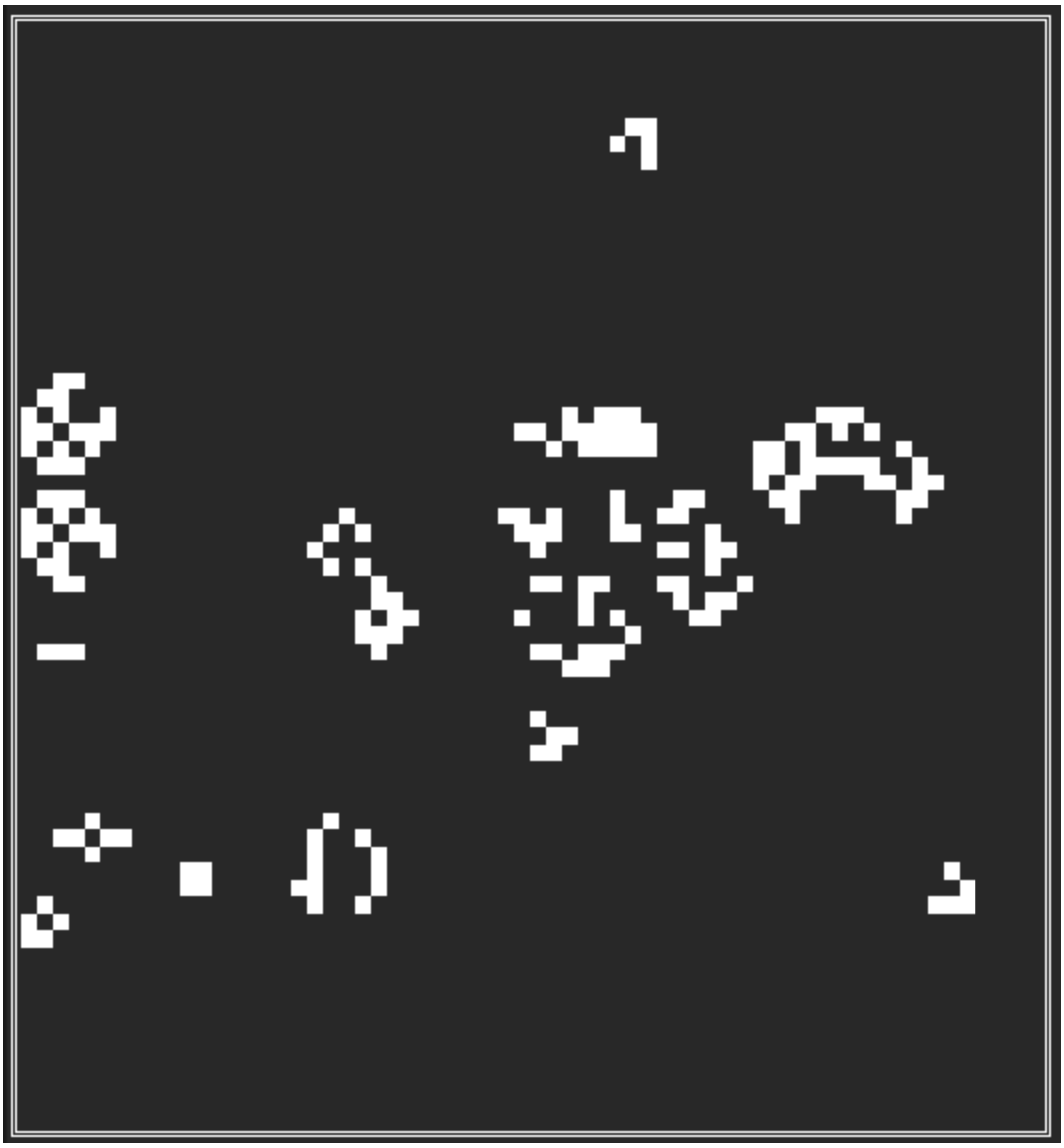
When drawing, I use half block and full block characters (▀, ▄, and █) so that one line of text represents two rows of cells. This results in a square board being displayed in dimensions much closer to square than otherwise, since terminal characters are taller than they are wide. When the size of the board is odd, there is an empty half-row at the bottom. I also use box-drawing characters (like ╔, ═, and ╗) to create a pretty-looking frame.

Alternative approaches

This parallelization is not incredibly sophisticated or optimized. Process 0 must do a great deal more work than other processes, as it must split, rejoin, and print the board, in addition to performing the same sub-board processing all other processes do. It's possible a manager/worker relationship would have a more even split of work, although that would require programming a special case for just 1 process.

Program output

Here is a screenshot from the program when ran with `mpirun -n 10 --oversubscribe conway 64 1200 -r -p` after several generations.



This is produced from the simple r-pentomino in the picture above.

Benchmarking

I created a bash file for benchmarking. It reads:

```
#! /bin/bash
echo "Q4" > "q4log.txt"
for p in {1..30}
do
     echo "P: $p" >> q4log.txt
     echo "P: $p" # so we can monitor progress
     for i in {1..20}
     do
          mpirun -n $p --oversubscribe conway 128 500 -D >> q4log.txt
     done
done
```

I chose to run the program with a 128x128 board, for 500 generations.

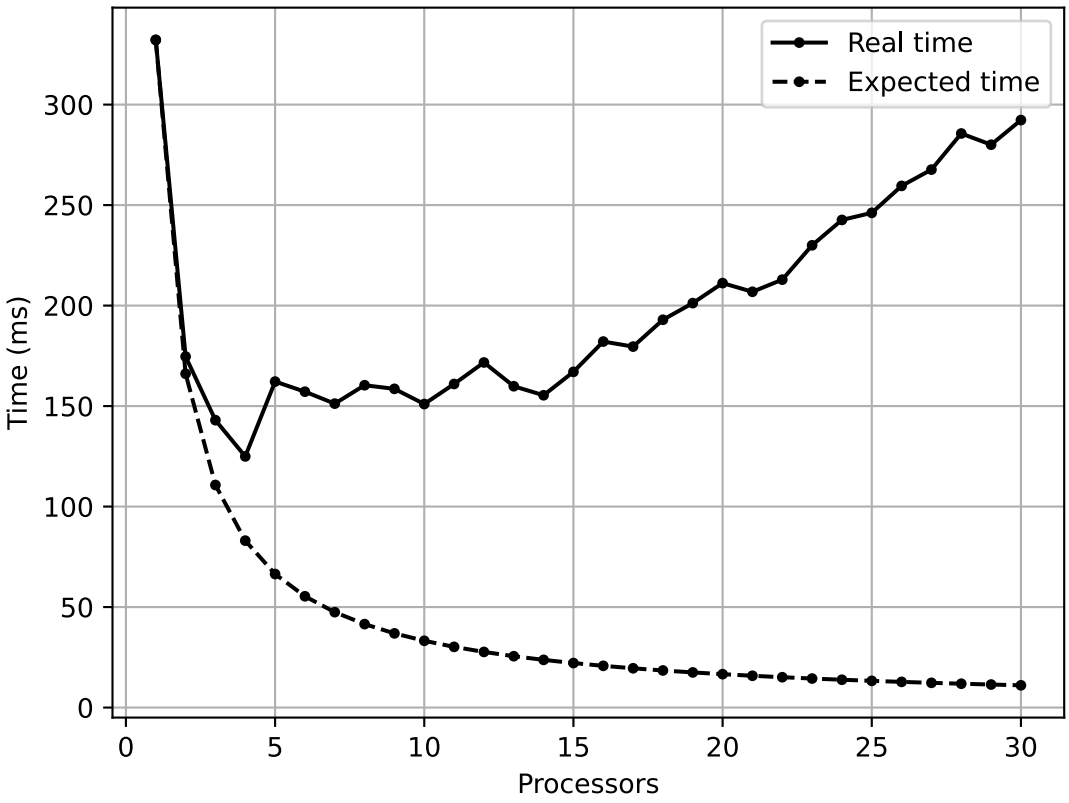
I ran the program 20 times for each process count. The output to `q4log.txt` looked like:

```
Q4
P: 1
T:   0.328004
T:   0.324502
T:   0.323949
T:   0.322222
T:   0.318287
T:   0.318352
T:   0.316238
T:   0.318758
T:   0.339853
T:   0.342758
T:   0.350925
T:   0.335636
T:   0.345871
T:   0.358558
T:   0.361306
T:   0.319137
T:   0.318341
T:   0.333645
T:   0.329426
T:   0.337955
P: 2
T:   0.168117
T:   0.177374
T:   0.172408
T:   0.169102
T:   0.168338
T:   0.176966
T:   0.174188
T:   0.172739
T:   0.182311

...
```

Benchmark results

To extract the results from the log file, and visualize them in a line graph, I wrote a python script. I used regex and matplotlib. This is my graph.



Takeaways

The performance gains are quite good when going from 1 to 4 processors. The real time for 2 processors is quite close to the theoretical time, which makes me smile.

It should not be too surprising that this program reaches peak performance at 4 processors, the number of CPUs my computer actually has. However, this is strikingly different than results from Q1 and Q2, which both hit a peak at 17 processors, far above my actual CPU count. However, Q1 and Q2 relied exclusively on `MPI_Reduce` for communication between processes, which may be why they parallelize effectively when oversubscribing. My `conway` program relies on `MPI_Isend`, `MPI_Irecv`, `MPI_Wait`, and `MPI_Bcast`, which may fare worse when oversubscribing. Note that `MPI_Bcast` is used exclusively for checking for quit when using `-p`, though.

I would certainly be very happy to benchmark my program on more real processors and more powerful computers, but I am of course limited by my own resources.